



A02 - Regresión Lineal: Gradiente Descendente y Pseudoinversa Algoritmos Metaheurísticos

Jose Luis Chavez Gomez

28 de agosto de 2026

1. Marco Teórico

1.1. Función Objetivo

En regresión lineal multivariable, el modelo predice:

$$\hat{y} = X\theta \quad (1)$$

La función de costo (error cuadrático medio) es:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \|X\theta - y\|^2 \quad (2)$$

1.2. Ecuaciones de Actualización de Gradiente Descendente

El gradiente de $J(\theta)$ es:

$$\nabla J(\theta) = \frac{1}{m} X^T (X\theta - y) \quad (3)$$

La regla de actualización iterativa:

$$\theta := \theta - \alpha \cdot \frac{1}{m} X^T (X\theta - y) \quad (4)$$

1.3. Pseudoinversa

Igualando el gradiente a cero se obtiene la ecuación normal:

$$\theta = (X^T X)^{-1} X^T y = X^+ y \quad (5)$$

Esta solución analítica evita el proceso iterativo de gradiente descendente, resolviendo directamente el sistema de ecuaciones normales mediante la pseudoinversa de Moore-Penrose, la cual generaliza el concepto de matriz inversa incluso cuando $X^T X$ no es invertible.

2. Implementación

2.1. Generación de Datos (dataset.py)

```
1 import numpy as np
2
3 def generar_datos(m=100, n=2, seed=42):
4     np.random.seed(seed)
5     X_raw = np.random.rand(m, n) * 10
6     X = np.hstack([np.ones((m, 1)), X_raw])
7     theta_real = np.array([5, 2, -3])
8     ruido = np.random.randn(m) * 1.0
9     y = X @ theta_real + ruido
10
11     return X, y, theta_real
```

Se generan $m = 100$ ejemplos con $n = 2$ variables independientes, más una columna de unos para el término de sesgo (bias). Los valores objetivo y se construyen a partir de un $\theta_{real} = [5, 2, -3]$ conocido, agregando ruido gaussiano para simular condiciones realistas.

2.2. Gradiente Descendente (gradient_descent.py)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from dataset import generar_datos
4
5 def objective_function(X, y, theta):
6     m = len(y)
7     J_theta = (np.sum((X @ theta - y) ** 2)) / (2*m)
8     return J_theta
9
10 def gradient_descent(X, y, theta_inicial, alpha, iteraciones):
11     m = len(y)
12     costos = []
13     theta = theta_inicial
14     for i in range(iteraciones):
15         gradiente = (1/m) * X.T @ (X @ theta - y)
16         theta = theta - alpha * gradiente
17         costos.append(objective_function(X, y, theta))
18     return theta, costos
19
20 if __name__ == "__main__":
```

```

21 X, y, theta_real = generar_datos()
22
23 costo = objective_function(X, y, theta_real)
24 print("Costo con theta_real:", costo)
25
26 theta_prueba = np.zeros(3)
27 costo_malo = objective_function(X, y, theta_prueba)
28 print("Costo con theta en ceros:", costo_malo)
29
30 theta_inicial = np.zeros(3)
31 theta_final, costos = gradient_descent(X, y, theta_inicial, alpha
    =0.01, iteraciones=10000)
32
33 print("Theta final:", theta_final)
34 print("Theta real:", theta_real)
35 print(" ltimo costo:", costos[-1])
36 plt.plot(costos)
37 plt.xlabel("Iteraci n")
38 plt.ylabel("J(theta)")
39 plt.title("Convergencia de Gradiente Descendente")
40 plt.savefig("convergencia_gd.png")
41 plt.show()

```

2.3. Pseudoinversa (pseudoinverse.py)

```

1 import numpy as np
2 from dataset import generar_datos
3 from gradient_descent import objective_function
4
5 def pseudoinversa(X, y):
6     theta = np.linalg.pinv(X) @ y
7     return theta
8
9 if __name__ == "__main__":
10     X, y, theta_real = generar_datos()
11
12     theta_pinv = pseudoinversa(X, y)
13     print("Theta pseudoinversa:", theta_pinv)
14     print("Theta real:", theta_real)
15
16     costo_pinv = objective_function(X, y, theta_pinv)
17     print("Costo con pseudoinversa:", costo_pinv)

```

2.4. Comparación (compare.py)

```

1 from dataset import generar_datos
2 from gradient_descent import gradient_descent, objective_function
3 from pseudoinverse import pseudoinversa
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 X, y, theta_real = generar_datos()

```

```

8
9 theta_gd, costos = gradient_descent(X, y, np.zeros(3), alpha=0.01,
    iteraciones=10000)
10 theta_pinv = pseudoinversa(X, y)
11 costo_pinv = objective_function(X, y, theta_pinv)
12
13 plt.plot(costos, label="Gradiente_Descendente")
14 plt.axhline(y=costo_pinv, color='r', linestyle='--', label="Pseudoinversa_
    ( ptimo )")
15 plt.xlabel("Iteraci n")
16 plt.ylabel("J(theta)")
17 plt.title("Convergencia_de_GD_vs._Pseudoinversa")
18 plt.legend()
19 plt.savefig("comparacion_gd_pinv.png")
20 plt.show()

```

Este script integra ambos métodos utilizando exactamente el mismo conjunto de datos, permitiendo comparar de forma justa el costo final alcanzado por cada uno.

3. Resultados

3.1. Experimento: Gradiente Descendente

Antes de ejecutar el algoritmo, se evaluó la función de costo con dos valores de referencia. Con $\theta_{real} = [5, 2, -3]$ (los parámetros usados para generar los datos sintéticos) se obtuvo $J(\theta) = 0,5065$, un valor cercano a la varianza del ruido gaussiano introducido, como es esperado. Con $\theta = [0, 0, 0]$ se obtuvo $J(\theta) = 57,0098$, confirmando que la función de costo distingue correctamente entre un modelo bien ajustado y uno arbitrario.

Tras ejecutar gradiente descendente con $\alpha = 0,01$ durante 10,000 iteraciones, se obtuvo:

$$\theta_{GD} = [4,772, 2,034, -2,965], \quad J(\theta_{GD}) = 0,4907 \quad (6)$$

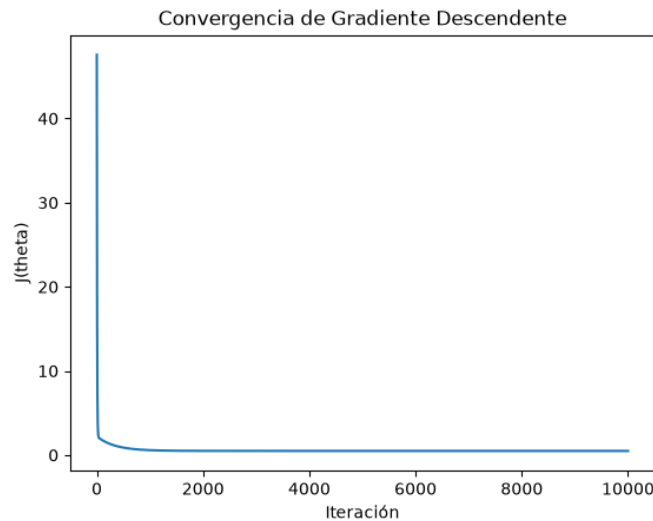


Figura 1: Convergencia del costo $J(\theta)$ a lo largo de las iteraciones

Como se observa en la Figura 1, el costo desciende abruptamente durante las primeras iteraciones y después se estabiliza, indicando que el algoritmo convergió correctamente antes de alcanzar las 10,000 iteraciones.

3.2. Experimento: Pseudoinversa

Al resolver el problema mediante la ecuación normal (pseudoinversa), se obtuvo:

$$\theta_{pinv} = [4,772, 2,034, -2,965], \quad J(\theta_{pinv}) = 0,4907 \quad (7)$$

Este resultado se obtiene en una sola operación matricial, sin necesidad de iterar ni de elegir una tasa de aprendizaje α .

3.3. Comparación

Método	θ_0 (bias)	θ_1	θ_2
θ_{real}	5.000	2.000	-3.000
Gradiente Descendente	4.772	2.034	-2.965
Pseudoinversa	4.772	2.034	-2.965

Cuadro 1: Comparación de parámetros obtenidos por cada método

Método	$J(\theta)$
θ_{real}	0.5065
$\theta = [0, 0, 0]$	57.0098
Gradiente Descendente	0.4907
Pseudoinversa	0.4907

Cuadro 2: Costo obtenido por cada método

Es importante notar que ambos métodos convergen a un valor ligeramente distinto de θ_{real} . Esto no representa un error, sino el comportamiento esperado: debido al ruido aleatorio agregado a los datos, el óptimo *muestral* (el mejor θ para esta muestra específica) no coincide exactamente con el θ_{real} usado para generarla. El hecho de que gradiente descendente y pseudoinversa converjan prácticamente al mismo valor confirma que ambos algoritmos encontraron correctamente ese óptimo.

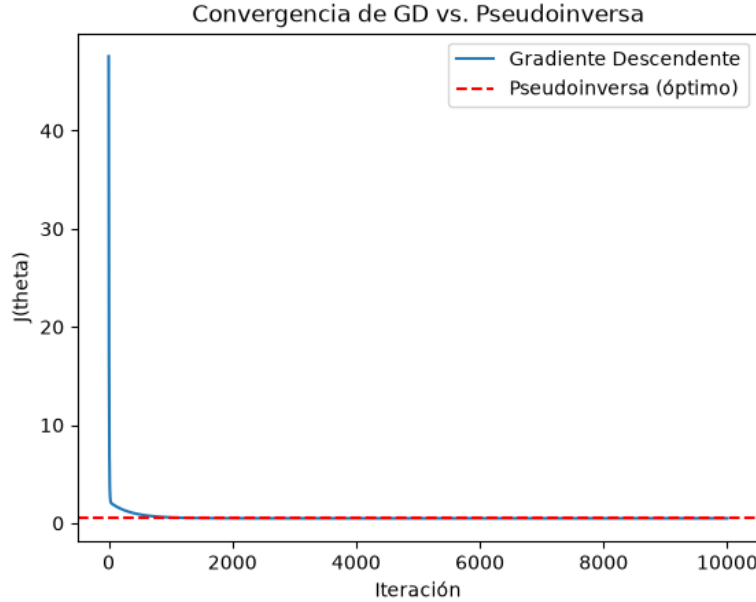


Figura 2: Comparación entre GD y la solución de la pseudoinversa

En la Figura 2 se observa cómo la curva de convergencia de gradiente descendente se aproxima asintóticamente a la línea horizontal que representa el costo óptimo obtenido mediante la pseudoinversa, evidenciando que ambos métodos convergen a la misma solución.

4. Conclusión

Se implementaron dos enfoques para resolver el problema de regresión lineal multivariable: un método iterativo (gradiente descendente) y un método analítico directo (pseudoinversa). Ambos métodos convergieron prácticamente al mismo valor de θ y al mismo costo final ($J(\theta) \approx 0,4907$), validando la correctitud de ambas implementaciones.

La principal diferencia observada entre ambos métodos es de naturaleza conceptual: gradiente descendente requiere elegir una tasa de aprendizaje α y un número de iteraciones, y su convergencia es progresiva, mientras que la pseudoinversa resuelve el problema en una sola operación matricial sin parámetros que ajustar. Sin embargo, esta ventaja de la pseudoinversa depende de que el problema sea lineal y de tamaño manejable, ya que el cálculo de $(X^T X)^{-1}$ o de la pseudoinversa puede volverse computacionalmente costoso para conjuntos de datos muy grandes o con muchas variables, escenario en el cual gradiente descendente resulta más práctico a pesar de su naturaleza iterativa.

5. Referencias

- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems* (3rd ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Strang, G. (2016). *Introduction to linear algebra* (5th ed.). Wellesley-Cambridge Press.